

FedSecureFormer: A Fast, Federated and Secure Transformer Framework for Lightweight Intrusion Detection in Connected and Autonomous Vehicles

Devika Sathyan, Vishnu Hari, Pratik Narang, *Senior Member, IEEE*, and Tejasvi Alladi, *Senior Member, IEEE*,
F. Richard Yu, *Fellow, IEEE*

Abstract—The era of Connected and Autonomous Vehicles (CAVs) reflects a significant milestone in transportation, improving safety, efficiency, and intelligent navigation. However, the growing reliance on real-time communication, constant connectivity, and autonomous decision-making raises serious cybersecurity concerns, particularly in environments with limited resources. This highlights the need for security solutions that are efficient, lightweight, and suitable for deployment in real-world vehicular environments. In this work, we introduce FedSecureFormer, a lightweight transformer-based model with 1.7 million parameters, significantly smaller than most encoder-only transformer architectures. The model is designed for efficient and accurate cyber attack detection, achieving 93.69% classification accuracy across 19 attack types and improving performance on 10 attack classes, outperforming several state-of-the-art (SOTA) methods. To assess its practical viability, we implemented the model within a Federated Learning (FL) setup using the FedAvg aggregation strategy. We also incorporated differential privacy to enhance data protection. For testing its generalization to unseen attacks, we used a histogram-guided GAN with LSTM and attention modules to generate unseen data, achieving 88% detection accuracy. Notably, the model achieved an inference time of 3.7775 milliseconds per vehicle on the Jetson Nano, approximately $100\times$ faster than SOTA models. These results position FedSecureFormer as a fast, scalable, and privacy-preserving solution for securing future intelligent transport systems.

Index Terms—Transformer, Attention, Federated Learning (FL), Differential Privacy (DP), Connected and Autonomous Vehicles (CAVs), Convolutional Neural Network (CNN), Long-Short-Term Memory (LSTM), Intrusion Detection System (IDS), Cybersecurity

I. INTRODUCTION

The National Association of Counties (NACo) stated that Connected Vehicles (CV) are “*Vehicles that can communicate with other vehicles, infrastructure, and devices through wireless network technology such as Wi-Fi and radio*”. According

to the US Department of Transportation, CVs utilize Vehicle-to-Everything (V2X) [1] communication to improve road safety, enable real-time data exchange, and ensure seamless interaction within highly dynamic vehicular networks by facilitating the exchange of critical information between vehicles, infrastructure, and networks [2]. Additionally, NACo defines Automated Vehicles (AV) (also known as driverless cars) as “*Vehicles equipped with technology that enables them to operate with little to no human assistance*.” AVs rely on integrated systems such as LiDAR, radar, cameras, and GPS [3] for safe navigation. The integration of Connected and Autonomous Vehicles (CAVs) [4] represents a significant advancement in transportation, with early milestones including Google’s self-driving car project, now known as Waymo. Intelligent Transportation Systems (ITS) further enhance traffic safety and efficiency through technologies like Artificial Intelligence, sensor networks, and V2X, particularly using IEEE 802.11p [4] for vehicular communication.

Despite their advantages, CAVs are vulnerable to a wide range of cybersecurity threats due to their reliance on interconnected systems, real-time data exchange, and wireless communication technologies [1]. These include Denial-of-Service (DoS) attacks, Sybil attacks, spoofing, and other similar attacks. Vulnerabilities often stem from communication protocols, such as Vehicle-to-Everything (V2X) and Controller Area Network (CAN) [5], as well as onboard components like LiDAR and GPS, due to the absence of encryption and authentication mechanisms.

Ensuring robust security measures has become essential as cyber threats in CAVs grow increasingly complex. This has led to the development of Intrusion Detection Systems (IDS) [2]. IDS solutions employ techniques such as machine learning [6], deep learning [4], and generative AI [2] to detect potential attacks in real time. As cyberattacks become more sophisticated, the role of IDS in reinforcing vehicular cybersecurity is increasingly critical. However, these models face severe drawbacks due to data imbalance problems, long-range dependencies in network traffic, and data poisoning. Unlike traditional models, transformers utilize self-attention mechanisms to capture complex dependencies and patterns within sequential data from network traffic.

Another key challenge is that existing IDS research for CAVs often prioritizes detection accuracy, with less emphasis on real-time response [5]. However, in dynamic environments, the ability to detect and respond to threats promptly is just as

Copyright (c) 20xx IEEE. Personal use of this material is permitted. However, permission to use this material for any other purposes must be obtained from the IEEE by sending a request to pubs-permissions@ieee.org. The work was supported by the Anusandhan National Research Foundation (ANRF), India under Grant ANRF/ARG/2025/008114/ENS.

Devika Sathyan, Vishnu Hari, Pratik Narang and Tejasvi Alladi are with the Department of Computer Science and Information Systems, BITS Pilani, Pilani Campus, 333031, India. (e-mail: p20210024@pilani.bits-pilani.ac.in; f20220094@pilani.bits-pilani.ac.in; pratik.narang@pilani.bits-pilani.ac.in; tejasvi.alladi@pilani.bits-pilani.ac.in).

F. Richard Yu is with the Department of Systems and Computer Engineering, Carleton University, Ottawa, ON K1S 5B6, Canada. (e-mail: richard.yu@carleton.ca).

critical as ensuring vehicle safety. Centralized models often struggle with non-IID (non-Independent and Identically Distributed) data and are prone to single points of failure, making them vulnerable to attacks. This limitation has led to the adoption of Federated Learning (FL) in the CAV domain [7], which enhances robustness by enabling distributed learning across multiple clients while improving adaptability to non-IID data through a client-server architecture.

To address the limitations identified in the existing literature, we developed a fast, lightweight transformer-based model specifically designed for intrusion detection in CAV environments. This direction has received limited attention so far. Although integrating FL with privacy-preserving mechanisms such as Differential Privacy (DP) or Homomorphic Encryption offers enhanced security, this approach remains underexplored in CAV-focused IDS research. In addition, most existing studies have overlooked the inference time in resource-constrained environments, even though a fast response is crucial for timely threat detection and decision-making.

The main contributions of this work are as follows:

- i **Lightweight Transformer Architecture:** We propose FedSecureFormer, a compact transformer model with only 1.7 million parameters, tailored for intrusion detection in Connected and Autonomous Vehicles (CAVs), achieving improved performance across ten out of 19 cyberattack classes.
- ii **Detection of Unseen Attacks:** To evaluate the model's generalization capability, we tested the model in a centralized setting using unseen adversarial sequences generated via a histogram-based GAN with LSTM and attention layers. FedSecureFormer achieved an 88% detection rate, demonstrating robust adaptability.
- iii **Federated and Privacy Preserving Deployment:** In contrast to the centralized evaluation, we deployed the model in a Federated Learning setup with FedAvg, showing less than 1.03% performance drop. With differential privacy, it achieved a 4.04% reduction in accuracy across 20 clients.
- iv **Real-Time Inference Capability:** FedSecureFormer achieved an inference time of 3.7775 milliseconds per vehicle on Jetson Nano, making it nearly 100× faster than existing models and well-suited for deployment in resource-constrained Intelligent Transport Systems.

The remainder of the paper is organized as follows. Section II reviews related work, while Section III outlines the background, including the CAV framework, and a taxonomy of vehicular misbehavior. Section IV details the proposed FedSecureFormer architecture in both centralized and federated settings. Section V describes the experimental setup, followed by results and analysis in Section VI. Finally, Section VII concludes the study.

II. LITERATURE SURVEY

Our literature review is structured as follows: We begin by delving into the IDS framework in the context of CAVs, then explore transformer-based approaches for intrusion detection. Next, we examine real-time response capabilities in resource-constrained environments, followed by techniques for detect-

ing previously unseen attacks, and conclude with a review of FL frameworks applied to IDS in CAV scenarios.

Intrusion detection within CAV environments involves identifying and categorizing misbehavior exhibited by participant vehicles. Sharma et al. [8] calculated plausibility scores related to the location and movement of vehicles, employing a confidence interval-based technique to identify physically improbable behaviors within the VeReMi dataset [9]. Whereas Sedar et al. [10] treated the intrusion detection task as a reinforcement learning problem, implementing an LSTM-based Q-network for classification. Moreover, Alladi et al. [1] deployed hybrid CNN-LSTM models on edge devices to facilitate rapid, effective intrusion detection.

Recent advancements have highlighted the effectiveness of transformer-based architectures in IDS for CAVs. Wang et al. [11] demonstrated strong detection performance using hierarchical vector transformers, which employed distinct self-attention mechanisms to capture inter-vehicular interactions and temporal dependencies separately. Similarly, Guan et al. [12] improved detection accuracy by integrating Principal Component Analysis (PCA) with multi-head self-attention, illustrating the value of hybrid transformer-based approaches. Additionally, Mundra et al. [13] proposed a lightweight transformer architecture in a decentralized framework by distributing the processing tasks and feature selection across multiple vehicles.

Computational latency and response time are critical considerations in CAV scenarios, where real-time feedback is essential to ensure safety and operational efficiency. Kumar et al. [5] reported promising results on the CarChallenge dataset [14] using a Raspberry Pi, achieving inference time below 50 microseconds. In contrast, Alladi et al. [1] documented a significantly higher average inference time across multiple models, at 234.505 milliseconds per vehicle, on a similar hardware setup, highlighting the wide variation in performance across different IDS implementations. Meanwhile, Mundra et al. [13] reported an average detection time of 22 to 25 milliseconds, further emphasizing the need for optimized, low-latency solutions in resource-constrained environments.

The effectiveness of an IDS is significantly strengthened by its ability to detect unseen attacks. Althunayyan et al. [3] proposed a two-stage architecture comprising an Artificial Neural Network (ANN) for detecting and classifying known attacks, and an LSTM autoencoder for identifying previously unseen attacks. Lin et al. [15] addressed visual spoofing attacks using a multimodal sensor fusion approach, introducing a multi-head self-attention-based fusion module to enhance robustness. Similarly, Fan et al. [16] developed a two-layer filtering mechanism that employed a One-Class SVM to detect unknown attacks and dynamically update the attack recognizer, which consists of a 1-D CNN augmented with a self-attention mechanism.

Liu et al. [17] proposed a decentralized FL framework that integrates a trust-based consensus mechanism using blockchain, where model updates are aggregated across multiple Roadside Units (RSUs), and evaluated using the KDD99 dataset. Bhavsar et al. [18] demonstrated a practical FL setup using Raspberry Pi clients and a Jetson Xavier server, evalu-

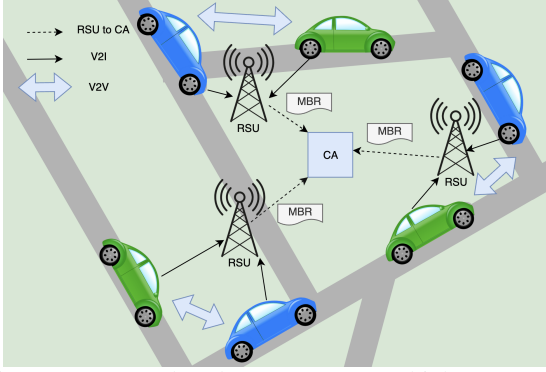


Fig. 1: Connected and Autonomous Vehicles Scenario

ating NSL-KDD and Car-Hacking datasets, with FedAvg and FedYogi aggregation strategies. Additionally, Al-Hawawreh et al. [19] introduced a distributed FL approach based on Augmented Lagrangian optimization, enabling each client to aggregate updates from its neighbouring clients, thereby enhancing collaboration in decentralized settings.

While prior studies such as [1], [8], [15], [16], [17], [18], and [19] have demonstrated strong detection performance, they fall short in addressing several essential aspects for a practical inter-vehicular IDS. Specifically, [1] has limited its evaluation to a subset of attack classes, leaving a significant portion unexamined, whereas [8] has used an older version of the VeReMi Extension dataset [9], which has comparatively fewer attack types. Although [16] addresses the detection of previously unseen attacks, it is based on an update mechanism that is prone to misclassifying known attack types, as acknowledged by the authors. Similarly, [15] introduced artificial modalities, but the work focused only on phantom spoofing attacks. In contrast, [17], [18], and [19] utilized FL; however, [17] used the obsolete KDD99 dataset, [18] restricted scalability to only four clients, and [19] relied on a centralized model with no consideration for deployment in resource-constrained environments.

Addressing these limitations collectively, we propose a lightweight IDS architecture optimized for deployment in resource-constrained environments. Despite the proven effectiveness of transformers across various security applications, their use in the vehicular domain, particularly in a protected FL setup, remains limited. This gap in current research motivates the development of a robust, scalable, and privacy-aware IDS tailored to the unique challenges of the CAV IDS.

III. PRELIMINARY BACKGROUND

1) *CAV Scenario*: A typical CAV scenario, as illustrated in Fig. 1, features Vehicle-to-Everything (V2X) communication, where a vehicle interacts with other vehicles and fixed infrastructure, such as traffic signals and road signage. Each vehicle has an Onboard Unit (OBU) that processes and fuses data from GPS and other onboard sensors. This information is transmitted to nearby RSUs, which are managed by the region's Control Authority (CA). RSUs are equipped with built-in mechanisms for detecting and classifying misbehavior with the installed IDS systems. When anomalous activity is

identified, the RSU generates a Misbehavior Report (MBR) and forwards it to the CA. The CA then evaluates the report and decides on appropriate actions, such as issuing alerts to other vehicles or revoking the offending vehicle's certificate.

2) *Dataset*: In this paper, we use the VeReMi Extension dataset [20], an enhancement of the original VeReMi dataset [9], which is a synthetically generated dataset incorporating a realistic sensor error model to reflect real-world scenarios better. The attack taxonomy is detailed in [2]. We have used nine features from the dataset: the timestamp (the time at which the message was sent) and the vehicle's motion coordinates (x and y), including position, speed, acceleration, and heading. The attack classes are as follows: A(0) - Normal, A(1) Constant position, A(2) - Constant position offset, A(3) - Random position, A(4) - Random position offset, A(5) - Constant speed, A(6) - Constant speed offset, A(7) - Random speed, A(8) - Random speed offset, A(9) - Eventual Stop, A(10) - Disruptive, A(11) - Data replay, A(12) - Delayed messages, A(13) - DoS, A(14) - DoS random, A(15) - DoS Disruptive, A(16) - Data replay sybil, A(17) - Traffic congestion sybil, A(18) - DoS random sybil and A(19) - DoS disruptive sybil.

3) *Transformers*: The Transformers, first introduced in [21], are deep neural networks that process sequential data effectively. Transformers replace conventional sequential models, such as RNNs and LSTMs, by parallelizing the processing of data sequences. Parallel processing in transformers is facilitated by a self-attention mechanism, which enables the capture of long-range dependencies in data sequences by learning the relationships between different data elements within the sequence. While initially introduced for Natural Language Processing (NLP) tasks, transformers have shown promising results in a wide range of domains, including temporal sequence modeling tasks [11, 12].

4) *Federated Learning*: Federated Learning (FL) [22] is a machine learning framework that can collaboratively train a shared model without exchanging data across different devices. Each round of FL involves training the model locally on clients (can be edge devices) and then sharing the model updates with the server. The final recombination of model updates is carried out using strategies such as the Federated Average Strategy (FedAvg) [22], a weight averaging technique, or the Federated Proximal Strategy (FedProx) [23], an iteration on FedAvg designed to improve its performance in heterogeneous environments. In this work, we have utilized FedAvg and FedProx strategies because, as stated in the existing literature [24, 25], both strategies exhibit similar complexities and convergence behavior in homogeneous environments. More recent studies [3, 7] demonstrate that FL can effectively enable a decentralized approach while also preserving privacy.

IV. PROPOSED INTRUSION DETECTION FRAMEWORK

The proposed FedSecureFormer architecture is illustrated in Fig. 2. FedSecureFormer is a transformer architecture specifically designed for IDS in the CAV scenario. The model was implemented in both centralized and distributed settings.

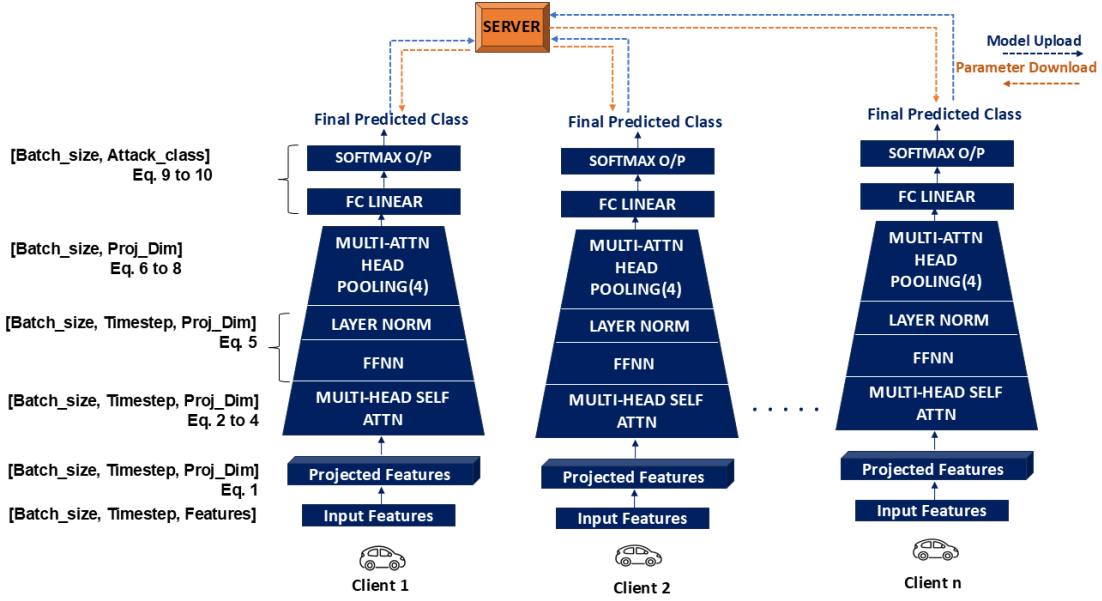


Fig. 2: FedSecureFormer architecture based on a 6-layer encoder-only transformer model, designed for efficient intrusion detection in CAVs, illustrated within a Federated Learning setup involving n clients.

A. Proposed FedSecureFormer Transformer

As shown in Fig. 2, the transformer architecture used in our work employs an encoder-only design with six stacked encoder layers. The input is first projected into an embedding space using a linear layer, followed by the addition of learnable absolute positional encodings to incorporate temporal context. Each encoder layer contains multi-head self-attention with two attention heads, each having its own Query, Key, and Value projections. This is followed by layer normalization and a two-layer Feed Forward Network (FFNN) with ReLU activation. Residual connections are applied after the attention and FFNN blocks to improve gradient flow and model stability. Post encoding, the output is passed through a multi-head attention pooling layer with four heads. This layer uses a multi-query strategy to share Keys and Values across heads. In contrast, each head maintains a unique learned Query vector, resulting in concatenated context vectors to form a pooled representation. Finally, this is passed through a fully connected linear layer and softmax to generate the final probability distribution over the target classes. The mathematical formulas used are as follows:

$$Z = \text{Stack}(\mathbf{z}_1, \dots, \mathbf{z}_t), \quad \text{where } \mathbf{z}_t = W_{\text{proj}} \cdot \mathbf{x}_t + \mathbf{b}_{\text{proj}} + \mathbf{p}_t \quad (1)$$

$$Q_i = ZW_i^Q, \quad K_i = ZW_i^K, \quad V_i = ZW_i^V \quad (2)$$

$$\text{head}_i = \text{softmax} \left(\frac{Q_i K_i^T}{\sqrt{d_k}} \right) V_i \quad (3)$$

$$\text{MHSA}(Z) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \cdot W_O \quad (4)$$

$$Z' = \text{LayerNorm}(Z + \text{FFN}(Z)) \quad (5)$$

where $\text{FFN}(Z) = \text{ReLU}(ZW_1 + b_1)W_2 + b_2$

$$q_j = u_j w_j^q, \quad k = Z' w^K, \quad v = Z' w^V \quad (6)$$

$$\text{context}_j = \text{softmax} \left(\frac{q_j k^T}{\sqrt{d_k}} \right) v \quad (7)$$

$$\text{pooled} = W_{\text{pool}} \cdot \text{Concat}(\text{context}_1, \dots, \text{context}_{H_{\text{pool}}}) \quad (8)$$

$$\mathbf{y} = W_{\text{cls}} \cdot \text{pooled} + \mathbf{b}_{\text{cls}} \quad (9)$$

$$\hat{\mathbf{p}} = \text{softmax}(\mathbf{y}), \quad \hat{c} = \arg \max(\hat{\mathbf{p}}) \quad (10)$$

$$\mathcal{L}_{\text{SmoothL1}}(\hat{\mathbf{p}}, \mathbf{y}) = \begin{cases} \frac{1}{2}(\hat{p}_k - y_k)^2, & \text{if } |\hat{p}_k - y_k| < 1 \\ |\hat{p}_k - y_k| - \frac{1}{2}, & \text{otherwise} \end{cases} \quad (11)$$

Here, the input feature vector $\mathbf{x}_t$ at timestep t is projected to d_{dim} using the weight matrix W_{proj} , bias $\mathbf{b}_{\text{proj}}$, and added to a learnable positional encoding $\mathbf{p}_t$ to produce the embedded vector $\mathbf{z}_t$. The complete input sequence is represented as Z . In the self-attention mechanism, Q_i , K_i , and V_i are the Query, Key, and Value matrices for attention head i , each of dimension d_k . The d_k is calculated by dividing d_{dim} by the number of attention heads H . The outputs from all heads are combined via an output projection W_O . Next, the output is fed to an FFN, followed by a layer normalization layer, resulting in Z' . Each head j uses a learned query vector q_j (utilizing a learnable vector u_j) in the multi-head pooling stage, while key k and value v from encoder outputs are shared across heads. The context_j learn q_j , k , v , and are concatenated and projected using W_{pool} to yield the final pooled vector, utilizing the pooling heads H_{pool} . For classification, the pooled vector is passed through a fully connected layer with weights W_{cls} and a bias vector $\mathbf{b}_{\text{cls}}$ to produce logits $\mathbf{y}$. The predicted probabilities $\hat{\mathbf{p}}$ are obtained via softmax, and the final predicted class $\hat{c}$ corresponds to the class with the highest probability. Finally,

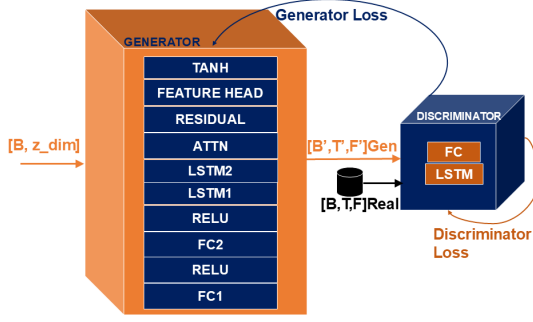


Fig. 3: His-AttnGAN architecture utilized for generating unseen data.

the loss is computed via smooth L1 loss using the predicted $\hat{p}_k$ and actual probabilities y_k .

B. Histogram-aware Attention GAN (Hist-AttnGAN)

To detect unseen attacks, we generated realistic samples using a histogram-guided GAN as depicted in Fig. 3. The generator component of this GAN comprises a two-layer LSTM architecture enhanced with multi-head self-attention utilizing four attention heads. Each timestep's output is subsequently passed through nine parallel feature projection heads—one for each feature—each composed of a small Multi-Layer Perceptron (MLP) consisting of combinations of Dense and LeakyReLU layers to independently generate feature values. The discriminator is constructed using a single-layer LSTM. During training, the generator receives input noise shaped $[Batch_size, Noise_Dimension]$ and produces outputs of shape $[Batch_size', Timesteps', Features']$. Both the generated data and the original training data, formatted identically as $[Batch_size, Timesteps, Features]$, are provided as inputs to the discriminator. The model training is guided by two primary objective functions: a discriminator loss, calculated using the Wasserstein distance coupled with a gradient penalty, and a generator loss that incorporates both adversarial loss and a histogram-based distribution loss.

$$\mathcal{L}_G = -\mathbb{E}_{\tilde{x} \sim \mathbb{P}_{\text{fake}}} [D(\tilde{x})] + \lambda_{\text{hist}} \cdot \sum_{i=1}^F \left\| \text{CDF}_{\text{real}}^{(i)} - \text{CDF}_{\text{fake}}^{(i)} \right\|_1 \quad (12)$$

$$\mathcal{L}_D = -\mathbb{E}_{x \sim \mathbb{P}_{\text{real}}} [D(x)] + \mathbb{E}_{\tilde{x} \sim \mathbb{P}_{\text{fake}}} [D(\tilde{x})] + \text{GP} \quad (13)$$

$$\text{GP} = \lambda_{\text{gp}} \cdot \mathbb{E}_{\hat{x} \sim \mathbb{P}_{\hat{x}}} \left[\left(\left\| \nabla_{\hat{x}} D(\hat{x}) \right\|_2 - 1 \right)^2 \right] \quad (14)$$

The generator loss $\mathcal{L}_G$ is calculated as the adversarial loss added with histogram loss, which is the Cumulative Distribution Functions' (CDF) L1 loss of the i^{th} real and fake features, respectively. The histograms are first constructed for each i^{th} real and fake feature and then normalized into probability distributions. These distributions are then converted into CDFs,

which are compared to measure the difference between the real and generated sequences. The λ_{hist} is a weighting factor for histogram loss. To maintain equal weightage for both adversarial loss and histogram loss, λ_{hist} was assigned a value of 1. $\mathcal{L}_D$ (discriminator loss) is the sum of Wasserstein's loss and gradient penalty (GP). GP is calculated using the L2 norm of the discriminator gradient, along with a penalty for deviation from the unit norm case. λ_{gp} is the Gradient penalty coefficient set to 10.

C. Federated Learning Framework with Differential Privacy

Our FL implementation aligns closely with the vanilla FL algorithm [7]. To deal with the class imbalance issue while splitting the data between multiple clients, we have computed the class weights (W_c) and used them as a parameter for the weighted cross-entropy loss ($\mathcal{L}_{\text{WCE}}$). The aggregation of model parameters across clients is performed using the FedAvg and FedProx strategies, with FedAvg yielding the best results.

$$\mathcal{L}_{\text{WCE}} = - \sum_{i=1}^C w_i \cdot y_i \cdot \log(\hat{y}_i) \quad (15)$$

$$w_{\text{global}} = \frac{1}{N} \sum_{i=1}^K n_i \cdot w_i \quad (16)$$

$$N = \sum_{i=1}^K n_i \quad (17)$$

$$M_{\text{global}} = \frac{1}{N} \sum_{i=1}^K n_i \cdot \text{metric}_i \quad (18)$$

$$\mathcal{L}_{\text{WCE}}(\theta) = \mathcal{L}(\theta) + \frac{\mu}{2} \left\| \theta - \theta^{(\ell)} \right\|^2 \quad (19)$$

Here C , y_i , $\hat{y}_i$, and w_i represent the number of attack classes, true class label, predicted class probability, and class weight for each class i , respectively. Whereas in the equation used to compute aggregation strategy, we have the following terms K , n_i , w_i , N , which represent the total number of clients, the number of training samples for each client, the local model parameter for each client c , and the total number of training samples across all clients. As shown in eq. 18, we calculated the average of all performance measures across all clients. In eq. 19, we calculated the aggregation using FedProx strategy, utilizing the weighted cross-entropy loss function ($\mathcal{L}_{\text{WCE}}(\theta)$), current local model parameters (θ), global model parameters received from the server before local training ($\theta^{(\ell)}$), and the FedProx constant (proximal term weight, μ).

To ensure privacy-preserving training in the federated environment, gradient updates are adjusted by applying gradient clipping and adding noise, as described below:

$$\tilde{g}_i = \frac{g_i}{\max \left(1, \frac{\|g_i\|_2}{C_{\text{clip}}} \right)} \quad (20)$$

$$\hat{g} = \frac{1}{B} \left(\sum_{i=1}^B \tilde{g}_i + \mathcal{N}(0, \sigma^2 C_{\text{clip}}^2 I) \right) \quad (21)$$

In the above equations, g_i denotes the original gradient of the i -th model parameter, while $\tilde{g}_i$ represents the clipped gradient. The clipping threshold C_{clip} restricts individual gradients to a specified maximum ℓ_2 norm. After gradient clipping, Gaussian noise $\mathcal{N}(0, \sigma^2 C_{clip}^2 I)$ is introduced to the averaged gradient (per batch B), with σ acting as the noise multiplier that controls the magnitude of the privacy-preserving noise. The resultant noisy gradient $\hat{g}_i$ guarantees DP by ensuring that gradient updates remain statistically indistinguishable, even when a single training sample is altered or removed. The privacy guarantee is formally denoted by two privacy budget parameters (ϵ, δ) , where ϵ refers to the maximum allowable privacy loss and δ refers to the probability of privacy failure with respect to ϵ , without compromising the model's utility (performance) [26].

The DP accountant utilized in our work is Rényi DP (RDP) [27, 28], and the privacy budget parameters (ϵ, δ) are calculated using the mechanism as described in [28, 29].

$$\epsilon(\alpha) = \frac{\alpha C_{clip}^2}{2\sigma^2} \quad (22)$$

$$\epsilon'(\alpha) = \frac{1}{\alpha - 1} \log \mathbb{E}_{k \sim \mathcal{B}(\alpha, \gamma)} [\exp((\alpha - 1) \cdot \epsilon_k)] \quad (23)$$

$$\epsilon_T(\alpha) = T \cdot \epsilon'(\alpha) \quad (24)$$

$$\epsilon(\delta) = \min_{\alpha > 1} \left(\epsilon_T(\alpha) + \frac{\log(1/\delta)}{\alpha - 1} \right) \quad (25)$$

Here, $\epsilon(\alpha)$ is the RDP parameter with sensitivity C_{clip} . α corresponds to the order of Rényi divergence, a tunable parameter with a value greater than 1, and we have used values typically ranging from 1.01 to 64. $\epsilon'(\alpha)$ provides the RDP after subsampling, where γ corresponds to the sampling probability, $\mathcal{B}(\alpha, \gamma)$ is the binomial distribution used in subsampling RDP analysis, and ϵ_k corresponds to the RDP parameter for the k -fold group. $\epsilon_T(\alpha)$ corresponds to the composition rule for T local rounds. The final eq. 25 shows the conversion from RDP to (ϵ, δ) -DP guarantee. The algorithm for the proposed FedSecureFormer is outlined as follows in Algorithms 1 and 2.

V. EXPERIMENTAL SETUP

This section provides details of the system specification, dataset specification, and model hyperparameters used in this study.

A. System Specification

All experiments were conducted on an NVIDIA RTX A6000 GPU within a software environment consisting of Python 3.10.12, PyTorch 2.6.0, CUDA 11.8, and the Visual Studio Code (VSCode) IDE. FL experiments were implemented using the Flower framework (version 1.14.0). The proposed FedSecureFormer model was also deployed on a Jetson Nano running JetPack 4.6.6 to demonstrate real-time inference and evaluate performance in resource-constrained environments.

Algorithm 1 FedSecureFormer Algorithm

```

1: Input:  $X \in \mathbb{R}^{T \times F}$  (input sequence), model weights  $\Theta = \{W_{proj}, \mathbf{b}_{proj}, \mathbf{p}_t, W_t^Q, W_t^K, W_t^V, W_O, W_{pool}, W_{cls}, \mathbf{b}_{cls}, q_i\}$ , shared keys  $K$ , values  $V$ 
2: Output: Predicted class  $\hat{c}$ , probability vector  $\hat{\mathbf{p}}$ 
3: Hyperparameters:  $T = 20$  (time steps),  $F = 9$  (features),  $H = 2$  (attention heads),  $H_{pool} = 4$  (pooling heads),  $d_k = 32$  (attention head dimension),  $d_{dim} = 64$  (projection dimension),  $C = 20$  (number of classes),
4:  $d_k = \frac{d_{dim}}{H}$ 
5: for  $t = 1$  to  $T$  do
6:    $\mathbf{z}_t = W_{proj} \cdot \mathbf{x}_t + \mathbf{b}_{proj} + \mathbf{p}_t$ 
7: end for
8:  $Z = \text{Stack}(\mathbf{z}_1, \dots, \mathbf{z}_T)$ 
9: for  $i = 1$  to  $H$  do
10:   $Q_i = ZW_i^Q$ 
11:   $K_i = ZW_i^K$ 
12:   $V_i = ZW_i^V$ 
13:   $\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i$ 
14: end for
15:  $\text{MHSA}(Z) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \cdot W_O$ 
16:  $\text{FFN}(Z) = \text{ReLU}(ZW_1 + b_1)W_2 + b_2$ 
17:  $Z' = \text{LayerNorm}(Z + \text{FFN}(Z))$ 
18: for  $j = 1$  to  $H_{pool}$  do
19:   $q_j = u_j w_j^Q$ 
20:   $k = Z' w_j^K$ 
21:   $v = Z' w_j^V$ 
22:   $\text{context}_j = \text{softmax}\left(\frac{q_j k^T}{\sqrt{d_k}}\right) \cdot v$ 
23: end for
24:  $\text{pooled} = W_{pool} \cdot \text{Concat}(\text{context}_1, \dots, \text{context}_{H_{pool}})$ 
25:  $\mathbf{y} = W_{cls} \cdot \text{pooled} + \mathbf{b}_{cls}$ 
26:  $\hat{\mathbf{p}} = \text{softmax}(\mathbf{y})$ 
27:  $\hat{c} = \arg \max(\hat{\mathbf{p}})$ 
28:  $\mathcal{L}_{\text{SmoothL1}}(\hat{p}_k, y_k) = \begin{cases} \frac{1}{2}(\hat{p}_k - y_k)^2, & \text{if } |\hat{p}_k - y_k| < 1 \\ |\hat{p}_k - y_k| - \frac{1}{2}, & \text{otherwise} \end{cases}$ 

```

B. Dataset Specification

The dataset used in this study is the VeReMi Extension dataset [20]. Prior studies [4, 30] have inspired us to use the sliding window mechanism with a window size of 20, where each sample captures 20 consecutive timesteps with nine features per timestep, resulting in an input shape of [20,9] for each sample. A slide length of 10 was used, resulting in lower repetition and a higher number of sequences. Any sequence shorter than 20 was discarded. Each sequence is associated with one of 20 distinct classes (A(0) to A(19)), representing various types of attack categories. The dataset was labeled and sequentially generated, producing the following sequence distribution per class: A(0) - 165373, A(1) - 3804, A(2) - 3793, A(3) - 3821, A(4) - 3701, A(5) - 3623, A(6) - 3886, A(7) - 3662, A(8) - 3705, A(9) - 3727, A(10) - 3776, A(11) - 3870, A(12) - 3745, A(13) - 12564, A(14) - 12103, A(15) - 12370, A(16) - 16981, A(17) - 3876, A(18) - 8122, and A(19) - 7654. The dataset was split into a 70:15:15 ratio for training, validation, and testing, respectively, ensuring that

Algorithm 2 FedSecureFormer in FL-DP Algorithm

```

1: Input:  $D = \bigcup_{i=1}^N D_i$  (federated dataset including  $X$  and  $y$ 
   features across  $N$  clients),  $R = 100$  (total rounds),  $E = 1$ 
   (epoch per round),  $B$  (batch size),  $\eta$  (learning rate),  $\sigma$ 
   (noise multiplier),  $C_{clip}$  (clipping norm),  $\Theta_0$  (initial global
   weights),  $T = 20$  (time steps),  $F = 9$  (features)
2: Output: Final global model  $\Theta_R$ 
3: Initialize FedSecureFormer weights  $\Theta_0$  on the server
4: for  $r = 1$  to  $R$  do
5:   Broadcast global weights  $\Theta_{r-1}$  to all clients  $i = 1$  to
      $N$ 
6:   for all client  $i = 1$  to  $N$  in parallel do
7:     Receive  $\Theta_{r-1}$ , set  $\Theta_i \leftarrow \Theta_{r-1}$ 
8:     for all batch  $(X, y) \in D_i$  do
9:        $\hat{y} \leftarrow \text{FedSecureFormer}(X; \Theta_i)$ 
10:       $\mathcal{L}_{\text{WCE}} = -\sum_{c=1}^C w_c \cdot y_c \cdot \log(\hat{y}_c)$ 
11:      Compute gradients:  $\nabla_{\theta_i} \leftarrow \nabla \mathcal{L}_{\text{WCE}}$ 
12:      Clip gradients:  $\nabla \theta_j \leftarrow \nabla_{\theta_i} / \max(1, \frac{\|\nabla_{\theta_i}\|_2}{C_{clip}})$ 
13:      Add noise and average:
         $\nabla \theta_j \leftarrow \frac{1}{B} \left( \sum_{i=1}^B \theta_j + \mathcal{N}(0, \sigma^2 C_{clip}^2 I) \right)$ 
14:      Update model:  $\Theta_i \leftarrow \Theta_i - \eta \cdot \nabla \theta_j$ 
15:     end for
16:     Return  $\Theta_i$  and metrics (Accuracy, F1, etc.)
17:   end for
18:   Aggregate:  $\Theta_r \leftarrow \frac{1}{\sum n_i} \sum n_i \cdot \Theta_i$ 
19: end for
20: return final model  $\Theta_R$ 

```

the original class imbalance was preserved.

C. Model Hyperparameters

The model took approximately 6937.16 seconds to train for 100 epochs using the Adam optimizer (learning rate = 0.0003). A gradient penalty of 10 and a latent space dimension of 100 were used. The hyperparameter choices for our transformer architecture were selected through an extensive ablation study (Section VI), resulting in six encoder layers, each equipped with two attention heads, and a multi-head attention pooling layer with four heads. In a centralized training setting, we set the projection dimension to 64, the batch size to 128, and used the smooth L1 loss. For the FL setup, we used a batch size of 64 and weighted cross-entropy loss to address class imbalance introduced by data splitting across multiple clients. The FL experiments used 20 clients across 100 rounds and 1 local epoch, with a clip norm of 5 and a noise multiplier of 0.001. For the His-AttnGAN, we considered using a batch size of 64 and a noise dimension of 128.

VI. PERFORMANCE EVALUATION AND ANALYSIS OF RESULTS

This section presents the numerical and graphical results of the proposed FedSecureFormer architecture. It includes an evaluation of attack-detection performance, comparisons with Transformer, TCN, and hybrid CNN-LSTM models, and an analysis of design choices such as encoder layers, attention

TABLE I: Comparison of Model Architectures – Hybrid CNN-LSTM and Temporal Convolutional Network against various Performance Measures

Model	Acc	Pre	Recall	F1-Score
1cnn1lstm	0.8780	0.7875	0.7801	0.7817
2cnn1lstm	0.8788	0.7942	0.7881	0.7821
2cnn2lstm	0.8777	0.7817	0.7771	0.7772
3cnn1lstm	0.8784	0.7890	0.7836	0.7840
2cnn1Bilstm	0.8781	0.7855	0.7807	0.7805
TCN	0.8918	0.8237	0.8180	0.8179

pooling, and attention heads. We also validated the FL with DP using 20 clients and demonstrated real-time inference on a Jetson Nano, confirming the model's suitability for resource-constrained environments.

A. Models Implemented to Perform IDS

We experimented with multiple models for attack detection:

- Hybrid CNN-LSTM: Models trained with varying numbers of CNN, LSTM, and BiLSTM layers with different loss functions.
- TCN: A basic Temporal Convolutional Network (TCN) model updated using cross-entropy loss.
- Transformers: Multiple transformer architectures explored by varying the number of layers, multi-head self-attention heads, and employing different loss functions.

B. Evaluation and Analysis of Results Obtained by Different Hybrid CNN-LSTM and TCN Models

Table I presents the performance metrics of the hybrid CNN-LSTM and TCN models. To ensure fairness, all the compared models discussed were evaluated under identical experimental settings. Among the hybrid configurations tested, the 2CNN-1LSTM (2cnn1lstm) architecture—comprising two convolutional layers followed by one LSTM layer achieved the best results when optimized using label smoothing loss. While other loss functions, such as focal and cross-entropy, were explored, label smoothing provided superior performance. Increasing the number of CNN or LSTM layers did not yield noticeable improvements in accuracy, but it significantly increased training time. The TCN model, however, delivered better overall performance due to its strength in modeling temporal dependencies. For clarity, only 2cnn1lstm and TCN models are compared in Table II. The TCN achieved an overall recall of 81.79%, outperforming 2cnn1lstm, which achieved a recall of 78.81%.

C. Evaluation and Analysis of Results obtained by Varying Transformer Architectures

As shown in Table III, the six-layer transformer encoder with two attention heads and multi-head attention pooling yielded the best results. Multi-head pooling outperformed mean pooling and single-head pooling in handling time-series data. As illustrated in Fig. 4, six layers offered the best trade-off between accuracy and complexity, while deeper models led to overfitting. Although four attention heads performed

TABLE II: Recall Comparison Between Hybrid CNN-LSTM and TCN Across Attack Type

Attack Class	Hybrid CNN-LSTM	TCN
A(0)	0.5099	0.5935
A(1)	0.8250	0.8697
A(2)	0.7300	0.7647
A(3)	0.9983	0.9967
A(4)	0.5917	0.9468
A(5)	0.9717	0.9559
A(6)	0.9617	0.9633
A(7)	0.9983	0.9942
A(8)	0.9983	0.9942
A(9)	0.8583	0.8529
A(10)	0.7917	0.8304
A(11)	0.4667	0.4128
A(12)	0.6333	0.7052
A(13)	0.9617	0.9739
A(14)	0.7400	0.7871
A(15)	0.5517	0.6623
A(16)	0.9733	0.9799
A(17)	0.4083	0.5186
A(18)	0.9183	0.8076
A(19)	0.8650	0.7498
Overall	0.7881	0.8179

TABLE III: Transformer Architecture Evaluation Across Pooling, Encoder and Attention Variants

Model	Pooling used	No of Enc/attn	Acc	Pre	Recall	F1-Score
Transformer	Mean Average	6 layer, 2 attn	0.9306	0.8786	0.8190	0.8392
		6 layer, 2 attn	0.9304	0.8756	0.8180	0.8382
		4 layer, 2 attn	0.9245	0.8671	0.8035	0.8273
	Multi-head attn	4 layer, 1 attn	0.9238	0.8626	0.8039	0.8260
		4 layer, 2 attn	0.9245	0.867	0.8035	0.827
		6 layer, 1 attn	0.9306	0.8761	0.8187	0.8380
		6 layer, 2 attn	0.9369	0.8798	0.8205	0.8404

slightly better, as illustrated in Fig. 5, we used two attention heads to reduce cost and retained four in the pooling layer for optimal performance. As reported in Table IV, the model achieved an accuracy of 93.69%, precision of 87.98%, recall of 82.05%, and F1 score of 84.04%. It performed exceptionally well across 10 attack types, with many achieving recall above 0.95 and perfect scores in 8 attack classes. We used a projection dimension of 64, which reduced model complexity and training time while resulting in only a 0.305% drop in performance compared to the larger variant with a 128-dimensional projection.

D. Comparison of Performance with SOTA

The recall comparison across attack classes shown in Table V indicates that our proposed model consistently outperforms traditional baselines and recent SOTA methods in terms of detection capability and reliability. It achieves the highest recall in 10 out of 19 attack classes, including perfect recall (100%) for critical classes such as A(3), A(7), A(8), and A(18). In contrast, models like Hybrid CNN-LSTM and prior works [2], [4], [6] exhibit lower and inconsistent performance. For example, [4] records only 5.5, 8, and 9% recall for classes A(2), A(4), and A(9), respectively. While [6] performs well

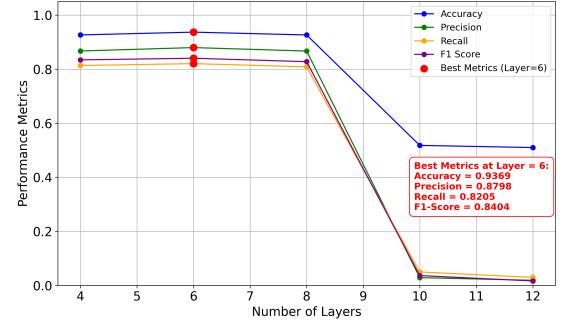


Fig. 4: FedSecureFormer achieves optimal performance across Accuracy, Precision, Recall, and F1-score plotted against the number of encoder layers. Red labels indicate the best values at six encoder layers.

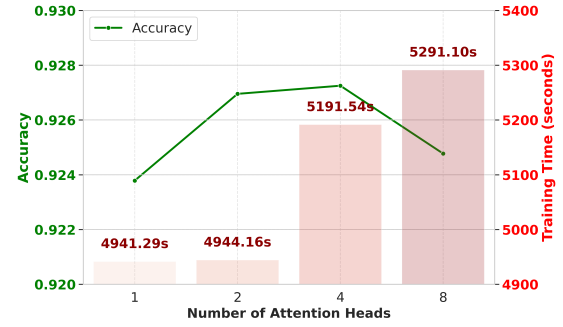


Fig. 5: FedSecureFormer achieves similar accuracy with both 2 and 4 attention heads, though 4 heads significantly increase training time.

on A(3), A(7), A(11), and A(12), it lacks consistency across other classes. Our model achieves 99.89% on A(1), 92.38% on A(2), 97.25% on A(5), and 99.94% on A(19), outperforming all baselines.

TABLE IV: Evaluation Metrics per Attack Type Utilizing Our Proposed Transformer

Attack Class	Accuracy	Precision	Recall	F1 Score
A(0)	0.9691	0.9694	0.9997	0.9843
A(1)	0.8295	0.9122	0.9514	0.9068
A(2)	0.9581	0.9038	0.6192	0.7349
A(3)	0.9935	0.9935	1.0000	0.9967
A(4)	0.8798	0.9765	0.8988	0.9361
A(5)	0.9715	0.9835	0.9876	0.9855
A(6)	0.9656	0.9896	0.9896	0.9825
A(7)	0.9973	0.9973	1.0000	0.9986
A(8)	0.9960	0.9960	1.0000	0.9980
A(9)	0.9509	0.9534	0.9214	0.6742
A(10)	0.9742	0.9048	0.8042	0.8515
A(11)	0.9318	0.4432	0.5297	0.4826
A(12)	0.9588	0.9977	0.5888	0.7406
A(13)	0.9385	0.9427	0.9952	0.9683
A(14)	0.9737	0.9837	0.7464	0.8488
A(15)	0.6348	0.8173	0.7397	0.7766
A(16)	0.9719	0.9943	0.9773	0.9857
A(17)	0.9218	0.4029	0.3209	0.3572
A(18)	0.9717	0.7238	1.0000	0.8352
A(19)	0.9514	0.6477	0.7133	0.6789
Overall	0.9369	0.8798	0.8205	0.8404

TABLE V: Comparison of Recall Rates for Various Attack Types with SOTA

Attack Class	Our Model	Hybrid CNN-LSTM	[2]	[4]	[6]
A(1)	0.9514	0.8250	0.9100	0.4250	0.8234
A(2)	0.6192	0.7300	0.4200	0.0550	0.7467
A(3)	1.0000	0.9983	0.7500	1.0000	0.9930
A(4)	0.8988	0.5917	0.4200	0.0800	0.9936
A(5)	0.9876	0.9717	0.9425	0.9450	0.8181
A(6)	0.9896	0.9617	0.9860	0.4200	0.0381
A(7)	1.0000	0.9983	0.9000	1.0000	0.9948
A(8)	1.0000	0.9983	0.9135	1.0000	0.9855
A(9)	0.9214	0.8583	0.9000	0.0900	0.0653
A(10)	0.8042	0.7917	1.0000	0.9850	0.0681
A(11)	0.5297	0.4667	1.0000	0.9850	0.9992
A(12)	0.5888	0.6333	0.6000	0.0850	0.9990
A(13)	0.9952	0.9617	1.0000	0.9800	0.7825
A(14)	0.7464	0.7400	1.0000	1.0000	0.7888
A(15)	0.7397	0.5517	0.4800	0.9950	0.6004
A(16)	0.9773	0.9733	0.8850	0.9850	0.0278
A(17)	0.3209	0.4083	1.0000	0.9750	0.5326
A(18)	1.0000	0.9183	0.9333	1.0000	0.7958
A(19)	0.7133	0.8650	0.7635	0.9900	0.7693

TABLE VI: Comparison of FedAvg Strategy in Different Settings

Strategy	# Clients	Epochs	Rounds	Acc	Pre	Recall	F1-score
Centralized	–	–	100	0.9369	0.8798	0.8205	0.8404
FedAvg	20	1	100	0.8301	0.9094	0.8457	0.8561
		2	50	0.8202	0.8960	0.8202	0.8439
		4	25	0.7551	0.8829	0.7551	0.7998

Even for challenging classes like A(11) and A(12), it maintains a competitive recall of 52.97%, and 58.88%, respectively, surpassing most benchmarks. Additionally, we also encountered the study [31], which utilizes the Mistral-7B model for attack detection in CAVs, employing 8 out of 19 attack classes from the VeReMi Extension dataset. However, the paper reported only class-averaged metrics, whereas ours shows an average improvement of approximately 0.0058 across all performance measures. Since the paper failed to report per-class metrics, a further detailed assessment is not possible.

E. Evaluation of Federated Learning Results

In the FL setup using FedSecureFormer, the best performance was observed with 20 clients as illustrated in Fig. 6. Each client received approximately 11350 training and 2830 test samples, with slight variations to simulate real-world traffic scenarios. Clients participated in a first-come, first-served manner. We used equivalent communication rounds (100) and local epochs (1) to align with centralized training. The FedAvg strategy yielded strong results with only a 1.03% performance drop compared to centralized training, as illustrated in Table VI. We also tested FedProx but observed

TABLE VII: FedProx Strategy Performance with Varying Clients and Proximal Parameters

Strategy	# Clients	Epochs	Rounds	Acc	Pre	Recall	F1	μ
FedProx	5	10	50	0.7841	0.6761	0.6761	0.7499	0.01
				0.6652	0.3798	0.3798	0.4150	0.1
	7			0.7541	0.4027	0.4027	0.4348	0.01
				0.6589	0.3201	0.3201	0.3086	0.1

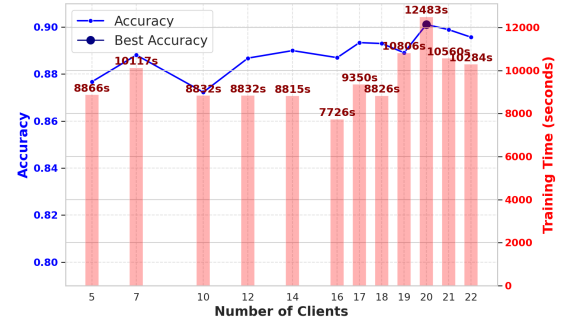


Fig. 6: Performance of FedSecureFormer in a federated setup, showcasing the ideal number of clients as 20.

TABLE VIII: Evaluation of Federated Learning under Various Differential Privacy Settings

Strategy	Noise Multiplier	Clip Norm	Acc	Pre	Recall	F1
FL with DP	0.1	1	0.441	0.791	0.441	0.501
	0.05	1	0.478	0.804	0.478	0.543
	0.5	1	0.278	0.635	0.278	0.320
	0.01	3	0.565	0.870	0.565	0.644
	0.001	1	0.731	0.907	0.731	0.789
	0.001	3	0.679	0.907	0.679	0.749
	0.001	5	0.827	0.898	0.749	0.797

inferior performance. As shown in Table VII, lower values of the proximal term μ yielded better performance. Additionally, results for 5 and 7 clients were evaluated using 10 local epochs and 50 communication rounds. Among these, the best performance was obtained with 5 clients, when using a μ value of 0.01. We have shown results only for 5 and 7 clients, which performed best among a subset of configurations selected from a range of 3 to 10.

F. Federated Learning Executed under Differential Privacy

Table VIII compares the performance of FL models with DP under different noise and clipping multipliers, using a batch size of 64. Performance dropped significantly when the noise multiplier exceeded 0.5, indicating sensitivity to noise. However, careful tuning enabled a balance between privacy and accuracy. The optimal DP-FL setting (noise = 0.001, clipping = 5) achieved an accuracy of 0.827, precision of 0.898, recall of 0.749, and an F1 score of 0.797, closely matching the non-private FL results with only a 4.04% performance loss. The privacy budget parameters (ϵ, δ) achieved with our model is (6.329, 1.00e-05). This shows that well-tuned DP-FL models can preserve privacy with minimal accuracy trade-off.

G. Detection of Unseen Attacks Using Hist-AttnGAN

The real and generated sequences are presented in Fig. 7, where the x-axis corresponds to the feature values across eight dynamic vehicle features and the y-axis represents the frequency of observations of each feature. The generated sequence exhibits narrow distributions, as our goal was not to introduce diversity but to create a new class of previously unseen data. The generated sequences were fed to the FedSecureFormer Transformer. We evaluated similarity thresholds

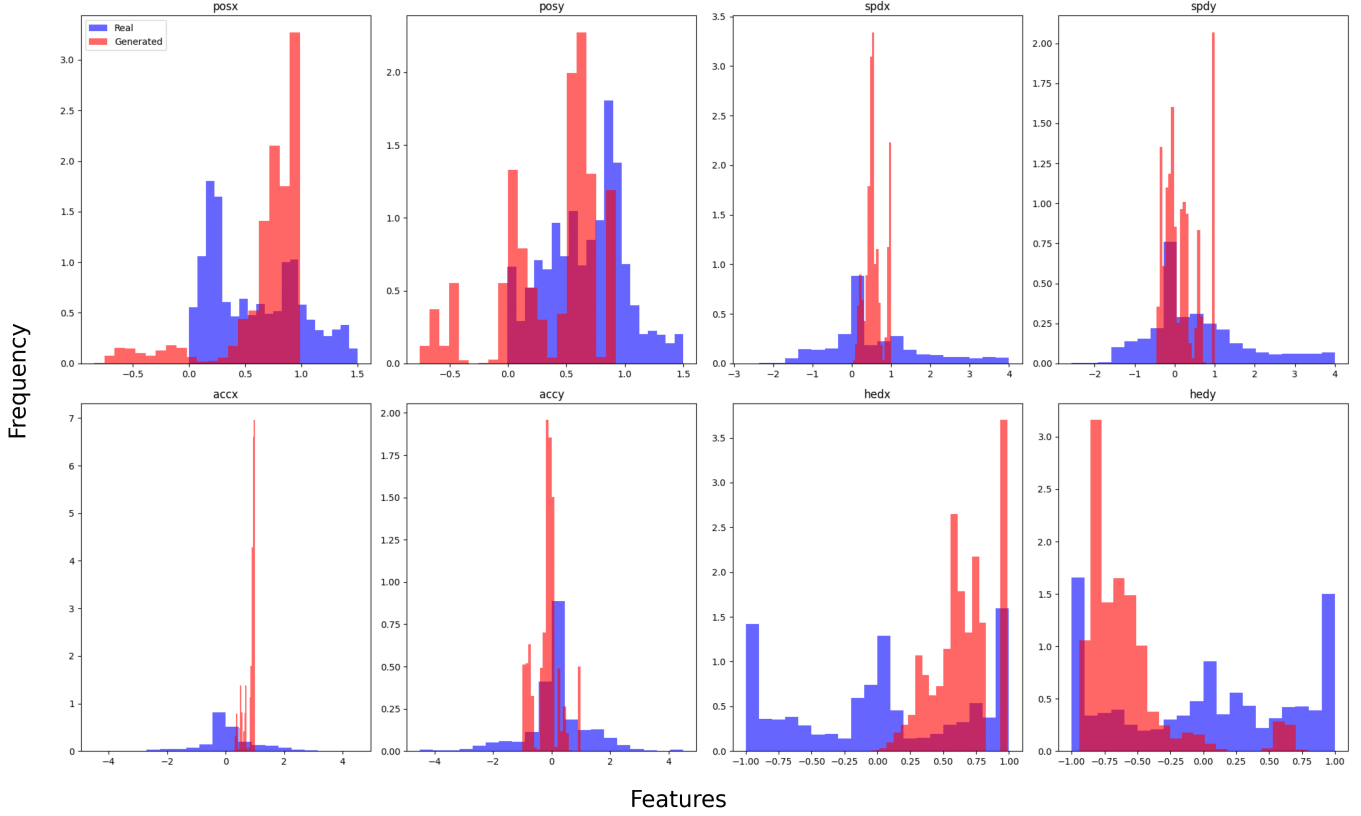


Fig. 7: Comparison of real and generated sequences plotted using histograms. Each subplot corresponds to one feature, with the x-axis representing feature values and the y-axis showing frequency counts.

ranging from 0.3 to 0.9 based on model confidence. We observed underfitting at thresholds below 0.5 and overfitting at thresholds above 0.5, making 0.5 the optimal similarity threshold. Using this optimal threshold of 0.5, we achieved a detection accuracy of 88%.

H. Comparison of Model Parameters with Encoder-Only Transformers

As illustrated in Fig. 8, the proposed FedSecureFormer has 1.70 million parameters, significantly fewer than those of all other compared encoder-only transformer models. The minimal architecture of our model features six encoder layers, two attention heads per encoder, multi-query multi-head attention pooling with four attention heads, a projection layer of 64 dimensions, trained on short sequences and dedicated for time series classification tasks, making it compact with fewer parameters.

I. Comparison of Inference Time with SOTA

Table IX illustrates the deployment of FedSecureFormer on a Jetson Nano for inference to demonstrate real-time applicability. The model achieved an average inference time of 3.7775 milliseconds per vehicle, highlighting its suitability for resource-constrained edge environments. Notably, our model outperforms existing SOTA approaches in terms of inference speedup. Specifically, FedSecureFormer achieves a speedup

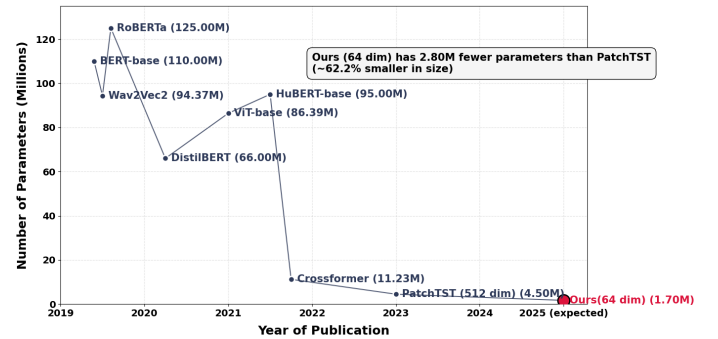


Fig. 8: Comparison of FedSecureFormer model parameters with other Encoder-only Transformers.

TABLE IX: Inference Time Comparison on Jetson Nano with SOTA

Reference	Environment	Data Setup	Prediction	Inference Time (ms)
[4]	Jetson Nano	0.07	511.82	511.89
[30]		0.33	287.74	288.07
[30]		0.32	251.24	251.56
Ours		0.0075	3.77	3.7775

of 135.53× compared to [4] and 66× compared to [30] when evaluated on the Jetson Nano hardware platform.

VII. CONCLUSION

In this work, we propose FedSecureFormer, a novel lightweight transformer model for cyberattack detection in Connected and Autonomous Vehicles (CAVs). The model features a six-layer encoder-only architecture with two attention heads, multi-query multi-head attention pooling, and a 64-dimensional projection. Comparable performance was observed between 64 and 128 dimensions, with the 64-dimensional version reducing the model size to 1.7 million parameters (approximately 1.9M fewer than the larger variant). Despite its compact design, the model achieved a classification accuracy of 93.69% across 19 attack categories. FedSecureFormer demonstrated strong resilience in real-world scenarios, with only a 1.03% drop in accuracy in the federated learning setup and a 4.04% performance loss under differential privacy. It also generalized well to unseen adversarial attacks generated using a histogram-based LSTM attention GAN, achieving 88% detection accuracy. Most importantly, it delivered real-time inference with an average of 3.7775 milliseconds per vehicle on a Jetson Nano, making it nearly 100× faster than state-of-the-art models and ideal for latency-sensitive Intelligent Transport Systems. We aim to explore the use of pre-trained transformer models to benchmark performance against our custom architecture. We also plan to investigate more efficient federated learning strategies that are mindful of bandwidth limitations, account for heterogeneous data distributions, minimize the performance gap between centralized and FL settings, and are better suited for deployment on edge devices.

REFERENCES

- [1] T. Alladi, V. Kohli, V. Chamola, F. R. Yu, and M. Guizani, "Artificial intelligence (ai)-empowered intrusion detection architecture for the internet of vehicles," *IEEE Wireless Communications*, vol. 28, no. 3, pp. 144–149, 2021.
- [2] D. S. R. R. Shrivastava, P. Narang, T. Alladi, and F. R. Yu, "Vadgan: An unsupervised gan framework for enhanced anomaly detection in connected and autonomous vehicles," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 9, pp. 12 458–12 467, 2024.
- [3] M. Althunayyan, A. Javed, and O. Rana, "A robust multi-stage intrusion detection system for in-vehicle network security using hierarchical federated learning," *Vehicular Communications*, vol. 49, p. 100837, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2214209624001128>
- [4] A. Chougule, V. Kohli, V. Chamola, and F. R. Yu, "Multibranch reconstruction error (mbre) intrusion detection architecture for intelligent edge-based policing in vehicular ad-hoc networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 11, pp. 13 068–13 077, 2022.
- [5] A. Kumar and T. K. Das, "Cavids: Real time intrusion detection system for connected autonomous vehicles using logical analysis of data," *Veh. Commun.*, vol. 43, no. C, Oct. 2023. [Online]. Available: <https://doi.org/10.1016/j.vehcom.2023.100652>
- [6] O. Slama, B. Alaya, S. Zidi, and M. Tarhouni, "Comparative study of misbehavior detection system for classifying misbehaviors on vanet," in *Proc. 8th International Conference on Control, Decision and Information Technologies (CoDIT)*, vol. 1. IEEE, 2022, pp. 243–248.
- [7] M. Z. Hossain, A. Imteaj, S. Zaman, A. R. Shahid, S. Talukder, and M. H. Amini, "Flid: Intrusion attack and defense mechanism for federated learning empowered connected autonomous vehicles (cavs) application," in *Proc. 2023 IEEE Conference on Dependable and Secure Computing (DSC)*, 2023, pp. 1–8.
- [8] P. Sharma and H. Liu, "A machine-learning-based data-centric misbehavior detection model for internet of vehicles," *IEEE Internet of Things Journal*, vol. 8, no. 6, pp. 4991–4999, 2021.
- [9] R. W. van der Heijden, T. Lukaseder, and F. Kargl, "Veremi: A dataset for comparable evaluation of misbehavior detection in vanets," 2018. [Online]. Available: <https://arxiv.org/abs/1804.06701>
- [10] R. Sedar, C. Kalalas, F. Vázquez-Gallego, and J. Alonso-Zarate, "Reinforcement learning based misbehavior detection in vehicular networks," in *Proc. IEEE ICC*, 2022, pp. 3550–3555.
- [11] H. Wang, J. Wang, Y. Jiao, S. Wang, and J. Guo, "Hierarchical vector transformer-based cyberattack detection for connected and autonomous vehicles via cloud platform," in *Proc. 2024 8th CAA International Conference on Vehicular Control and Intelligence (CVCI)*, 2024, pp. 1–6.
- [12] Q. Guan, T. Zhang, Y. Qin, Y. Zhou, Y. Zhu, Y. Zhong, X. Huang, Z. Duan, Z. Li, C. Liu, and X. Wu, "Transformer model with multi-type classification decisions for intrusion attack detection of track traffic and vehicle," in *Proc. ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 4510–4514.
- [13] A. Mundra, P. Vyas, and V. K. Verma, "Decentralized and lightweight transformer-based framework for cybersecurity in vehicular networks," *Engineered Science*, 2025.
- [14] H. Kang, B. I. Kwak, Y. H. Lee, H. Lee, H. Lee, and H. K. Kim, "Car hacking and defense competition on in-vehicle network," in *Proc. Workshop on Automotive and Autonomous Vehicle Security (AutoSec)*, vol. 2021. NDSS San Diego, CA, 2021, p. 25.
- [15] F. Lin, H. Yan, J. Li, Z. Liu, L. Lu, Z. Ba, and K. Ren, "Phade: Practical phantom spoofing attack detection for autonomous vehicles," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4199–4214, 2024.
- [16] C. Fan, J. Cui, H. Jin, H. Zhong, I. Bolodurina, and D. He, "Auto-updating intrusion detection system for vehicular network: A deep learning approach based on cloud-edge-vehicle collaboration," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 10, pp. 15 372–15 384, 2024.
- [17] H. Liu, S. Zhang, P. Zhang, X. Zhou, X. Shao, G. Pu, and Y. Zhang, "Blockchain and federated learning for collaborative intrusion detection in vehicular edge computing," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 6, pp. 6073–6084, 2021.
- [18] M. Bhavsar, Y. Bekele, K. Roy, J. Kelly, and D. Limbrick, "FI-ids: Federated learning-based intrusion detection system using edge devices for transportation iot," *IEEE Access*, 2024.
- [19] M. Al-Hawawreh and M. S. Hossain, "Federated learning-assisted distributed intrusion detection using mesh satellite nets for autonomous vehicle protection," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 854–862, 2023.
- [20] J. Kamel, M. Wolf, R. W. van der Hei, A. Kaiser, P. Urien, and F. Kargl, "Veremi extension: A dataset for comparable evaluation of misbehavior detection in vanets," in *Proc. IEEE ICC*, 2020, pp. 1–6.
- [21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [22] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," 2023. [Online]. Available: <https://arxiv.org/abs/1602.05629>
- [23] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," 2020. [Online]. Available: <https://arxiv.org/abs/1812.06127>
- [24] S. W. Herlambang, F. Dewanta, and Y. Purwanto, "Federated learning approaches for iot intrusion detection based on fedavg and fedprox on iid and non-iid data," in *2025 International Conference on Information and Communication Technology (ICoICT)*. IEEE, 2025, pp. 1–6.
- [25] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," in *International conference on machine learning*. PMLR, 2020, pp. 5132–5143.
- [26] S. De, L. Berrada, J. Hayes, S. L. Smith, and B. Balle, "Unlocking high-accuracy differentially private image classification through scale," *arXiv preprint arXiv:2204.13650*, 2022.
- [27] I. Mironov, "Rényi differential privacy," in *2017 IEEE 30th computer security foundations symposium (CSF)*. IEEE, 2017, pp. 263–275.
- [28] Y.-X. Wang, B. Balle, and S. P. Kasiviswanathan, "Subsampled rényi differential privacy and analytical moments accountant," in *The 22nd international conference on artificial intelligence and statistics*. PMLR, 2019, pp. 1226–1235.
- [29] I. Mironov, K. Talwar, and L. Zhang, "Rényi differential privacy of the sampled gaussian mechanism," *arXiv preprint arXiv:1908.10530*, 2019.
- [30] T. Alladi, V. Kohli, V. Chamola, and F. R. Yu, "A deep learning based misbehavior classification scheme for intrusion detection in cooperative intelligent transportation systems," *Digital Communications and Networks*, vol. 9, no. 5, pp. 1113–1122, 2023.
- [31] W. Hamhoun and S. Cherkaoui, "Mistralbsm: Leveraging mistral-

7b for vehicular networks misbehavior detection,” *arXiv preprint arXiv:2407.18462*, 2024.